Design Project: Pendulum

by

Austin Chung, Nathaniel McIntire, Nawedullah Akbari

Submitted to the Department of Mechanical and Aerospace Engineering in partial fulfillment of the requirements for the course of MAE 435: Principles of Automatic Control

at the

NORTH CAROLINA STATE UNIVERSITY

November 2025

Author ....

Nathaniel McIntire, Nawedullah Akbari, Austin Chung

Department of Mechanical and Aerospace Engineering

December 2, 2025

Submitted to ....

Dr. Fen Wu

Professor

1. Introduction

The main goal of this project is to design and analyze a feedback control system for a rotary pendulum. The rotary pendulum is operating around a non-zero equilibrium angle. Instead of stabilizing the pendulum at its natural hanging position, the objective of this project is to hold it at an elevated angle of $\Theta = \frac{\pi}{3}$ radians and design it to track a slight 10° change in position with high accuracy. To achieve the desired requirements, the nonlinear pendulum dynamics are first linearized about the chosen operating point, yielding a simplified model suitable for controller design. Using the linearized transfer function, we developed a compensator to meet specific performance requirements, including fast settling time, minimal overshoot, and minimal steady-state error. The root locus method was applied to tune a lead controller for the desired transient behavior, followed by a lag controller if additional steady-state accuracy is required. Once the controller is finalized, it is implemented in both the linear Simulink model and the provided nonlinear pendulum simulation to verify the performance of our system. This approach allows the system's response to be analyzed under ideal and realistic conditions, demonstrating how feedback control can precisely regulate the pendulum's motion around a non-zero equilibrium.

2. System Modeling

To design a feedback control system capable of regulating the pendulum at a nonzero equilibrium angle, an accurate mathematical model of the system is required. This section develops both the nonlinear and linearized representations of the pendulum dynamics. First, the nonlinear equations of motion are presented to understand the true behavior of the system. Next, the equilibrium torque required to hold the pendulum at the desired equilibrium point is computed. Finally, the nonlinear model is linearized about this equilibrium to produce a state-space model, which is suitable for control design methods, such as root locus, and compensator analysis discussed in sections 3 and 4.

2.1 Nonlinear System Dynamics

The pendulum system can be modeled as a single rigid body actuated by an applied torque T_c. The nonlinear state-space equation of the simple pendulum is:

$$\begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \end{bmatrix} = \begin{bmatrix} x_2 \\ -\frac{g}{l}sin(x_1) + \frac{T_c}{ml^2} \end{bmatrix}$$

Where $x_1 = \theta$ is the pendulum angle (rad), $x_1 = \dot{\theta}$ is the angular velocity (rad/s), $m = 1\ kg$, $l = 0.5\ m$, $g = 9.81\ m/s^2$, and T_c is the input torque (N*m).

2.2 Equilibrium Point Calculation

To design a linear controller, the system must first be linearized about a fixed operating point.

The desired equilibrium angle of: $\overline{\theta} = \frac{\pi}{3}$ and $\overline{\dot{\theta}} = 0$

At equilibrium, accelerations are zero, thus $\dot{x}_2 = 0$. Using the non-linear equation:

$$\overline{x} = \begin{bmatrix} \overline{\dot{x}_1} \\ \overline{\dot{x}_2} \end{bmatrix} = \begin{bmatrix} \overline{\theta} \\ 0 \end{bmatrix} = \begin{bmatrix} \frac{\pi}{3} \\ 0 \end{bmatrix}$$

$$0 = -\frac{g}{l}sin(\overline{\theta}) + \frac{\overline{T_c}}{ml^2}$$

Substituting these equilibrium conditions into the nonlinear system allowed you to solve for the torque $\overline{T_c}$ required to hold the pendulum at the desired angle. Further calculation is shown step by step on page 1, in the uploaded derivation and results in:

$$\overline{T_c} = mlgsin(\frac{\pi}{3}) = 4.248\ N * m$$

2.3 Linearization

The nonlinear dynamics were linearized around $(\overline{x}, \overline{u})$, as shown in pages 2-3 of the handwritten calculations. Slight deviations from the equilibrium are defined as:

$$\hat{x}_1 = x_1 - \overline{x_1}, \hat{x}_2 = x_2 - \overline{x_2}, \hat{u} = T_c - \overline{T_c}$$

We then computed the Jacobians:

$$A = \frac{\partial f}{\partial x}|_{(\overline{x},\overline{u})}, B = \frac{\partial f}{\partial u}|_{(\overline{x},\overline{u})}, C = \frac{\partial g}{\partial x}|_{(\overline{x},\overline{u})}, D = \frac{\partial g}{\partial u}|_{(\overline{x},\overline{u})}$$

$$A = \begin{bmatrix} 0 & 1 \\ -\frac{g}{l}cos(\overline{x_1}) & 0 \end{bmatrix}, B = \begin{bmatrix} 0 \\ \frac{1}{ml^2} \end{bmatrix}, C = \frac{\partial g}{\partial x}\begin{bmatrix} \frac{\partial g}{\partial x_1} & \frac{\partial g}{\partial x_2} \end{bmatrix}, D = \frac{\partial g}{\partial u}$$

Evaluating them at the equilibrium point yields:

$$A = \begin{bmatrix} 0 & 1 \\ -9.81 & 0 \end{bmatrix}, B = \begin{bmatrix} 0 \\ 4 \end{bmatrix}, C = [1 \ \ 0], D = 0$$

These matrices define the linearized state-space model used for the rest of the control design and simulation.

3. Open-loop analysis

Before designing a controller, it is important to understand the open-loop behavior of the linearized pendulum system. This section derives the transfer function from the linear state-space model and analyzes the system's natural response to a unit-step input. The open-loop characteristics show how the system behaves without a controller's input, and why the controller is necessary. It also shows what exactly we need to include in our design process to find the most optimal controller.

3.1 Transfer function

To obtain the transfer function of the linearized pendulum system, we begin by using a state-space representation derived in part 1:

$$\dot{\widehat{x}} = \begin{bmatrix} 0 & 1 \\ -9.81 & 0 \end{bmatrix}\widehat{x} + \begin{bmatrix} 0 \\ 4 \end{bmatrix}\widehat{u}, \quad \widehat{y} = [1 \ \ 0]\widehat{x}$$

Writing the Laplace-domain state equation, assuming zero initial conditions:

$$s\widehat{X}(s) = A\widehat{X}(s) + B\widehat{U}(s)$$

$$\widehat{X}(s) = (sI - A)^{-1}B\widehat{U}(s)$$

Computing for $(sI - A)$:

$$sI - A = \begin{bmatrix} s & -1 \\ -9.81 & s \end{bmatrix}$$

Computing the inverse yields:

$$(sI - A)^{-1} = \frac{1}{s^2 + 9.81}\begin{bmatrix} s & -1 \\ -9.81 & s \end{bmatrix}$$

With further calculation, as shown in pages 4-5 of the handwritten calculation, yields an output equation:

$$\widehat{Y}(s) = C\widehat{X}(s) = C(sI - A)^{-1}B\widehat{U}(s)$$

Multiplying by $C = [1 \ \ 0]$:

$$\widehat{Y}(s) = \frac{4}{s^2+9.81}\widehat{U}(s)$$

The final transfer function:

$$G(s) = \frac{\widehat{\theta}}{\widehat{T_c}(s)} = \frac{4}{s^2+9.81}$$

This transfer function represents the linearized relationship between minor deviations in input torque and the resulting small deviations in pendulum angle about the equilibrium point $\theta = \frac{\pi}{3}$.

3.2 Unit-Step Response

The open-loop transfer function found represents a second-order system, with no damping ratio and a natural frequency of:

$$\omega n = \sqrt{9.81} \approx 3.13 \text{ rad/s} \approx 179.456 \text{ deg/s}$$

To characterize its open-loop behavior, a unit-step torque input was applied:

$$T_c(\text{t}) = 1 \text{ N\textbackslash m}$$

Which gives a closed-form response:

$$\theta(t) = \frac{4}{9.81}(1 - cos(\sqrt{9.81}t))$$

And using the Final Value Theorem:

$$\theta(t)_{ss} = \frac{4}{9.81} \approx 0.408\, rad \approx 23.377\, deg$$

However, since the system is undamped, there is no settling time, and the steady-state value from the final value theorem gives the mean value, and the system outputs a constant oscillatory motion, and therefore cannot satisfy any transient performance requirements.

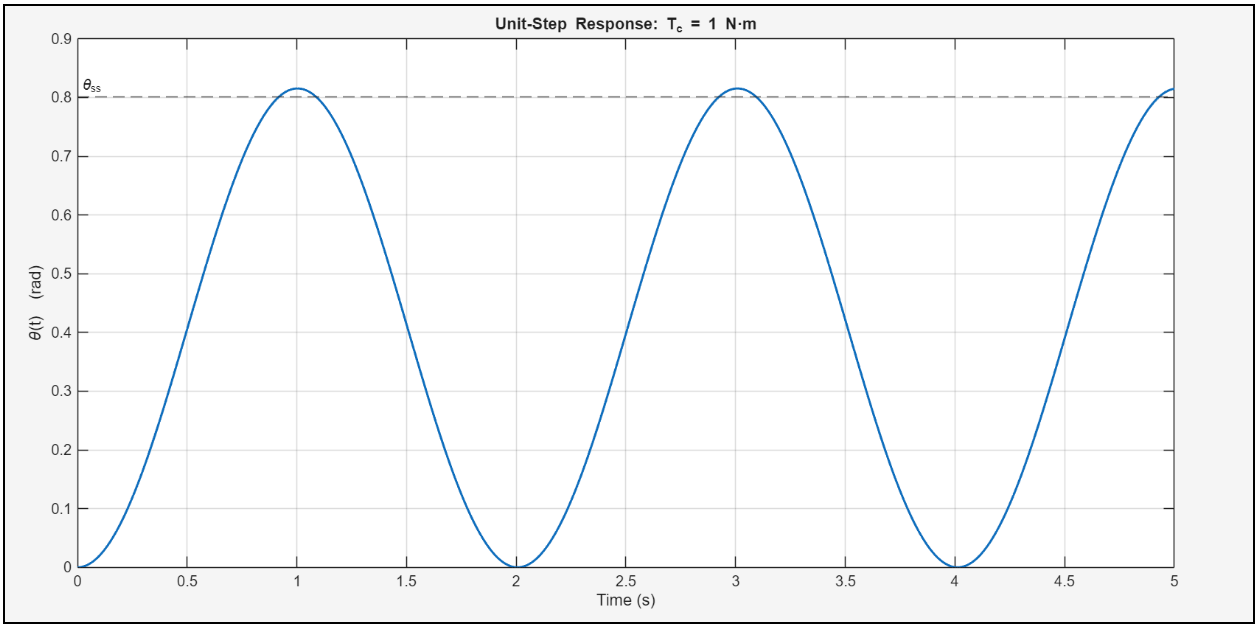

Figure 1: Unit-Step Response for Open-Loop Transfer Function

As shown in the figure, the peak is around 0.81 which matches the max $\theta(t)_{ss}$ value when t = 1. The system oscillates forever, and therefore a lead compensator is required to introduce damping and to meet the specified transient performance requirements.

3.3 Root Locus Method

Utilizing techniques from the Root Locus Method, we were able to come up with a region where we could put our closed loop poles. Below are the relevant mathematical relationships utilized to come up with our graphical constraints.

$$M_p = e^{-\frac{\pi\zeta}{\sqrt{1-\zeta^2}}} = 0.05 \Rightarrow \zeta = \sim 0.690$$

$$cos(\theta) = \zeta \Rightarrow \theta = cos^{-1}(\zeta) = \sim 46.370^o$$

$$T_s = \frac{4}{\sigma} = 0.5 \Rightarrow \sigma = 8 \qquad \text{(make this negative)}$$

Below in Figure 2 is the handwork used in our Root Locus analysis. Poles at -9±9j were attempted, however, this pole location did not satisfy our angle requirement. Attempts of poles at -10±4j were tried computationally, however this did not satisfy our angle requirements either. Therefore, a MATLAB iterative script was used for the bulk of computational work.

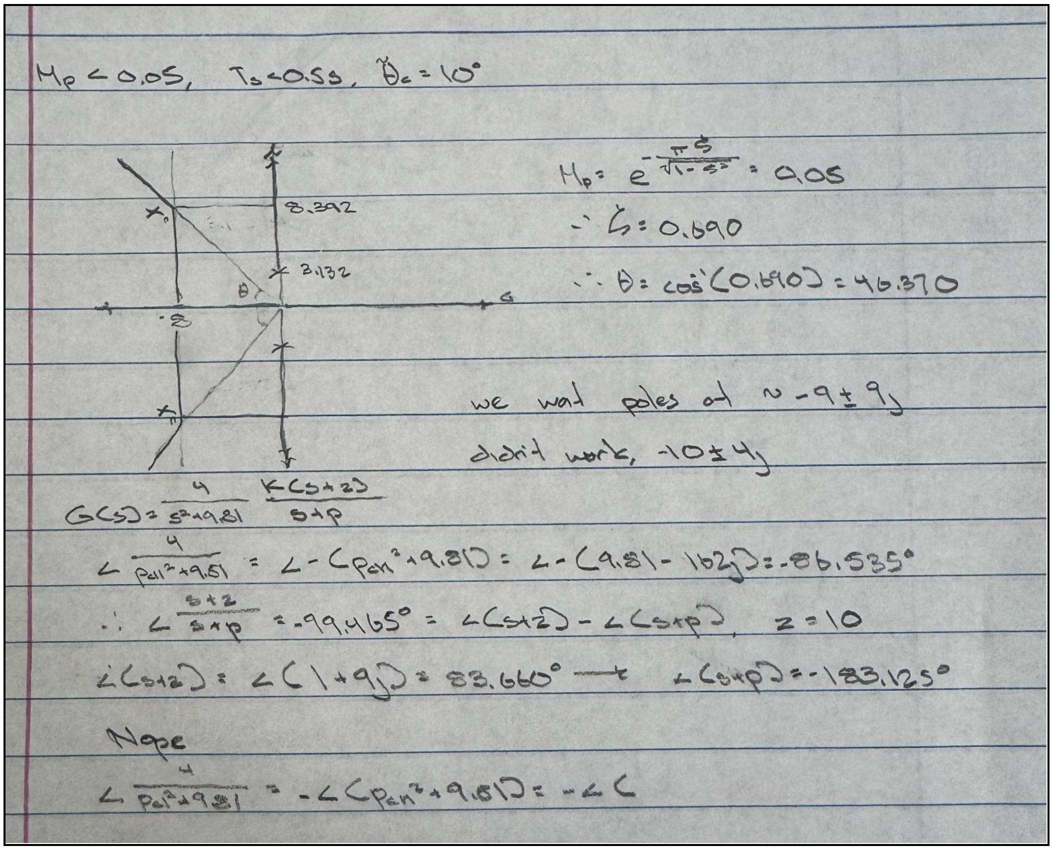

Figure 2: Root Locus Method Handwork

4. Controller Design

This section focuses on designing a controller that can stabilize the pendulum system, and meet the required specifications listed in section 4.1. Using the linearized model derived earlier, root locus methods are utilized to select appropriate compensator values.

4.1 Control Requirements

The lead compensator must be designed using the root locus method to achieve the transient performance with overshoot less than 5% and settling time less than 0.5 sec for a step command $\theta_c = 10° = \frac{\pi}{18} rad$. Finally, the design must reduce the unit-step steady-state tracking error less than 0.01° for the $10°$ step input.

4.3 Lead Compensator Design

A lead compensator of the form

$$G_c(s) = K\frac{s+z}{s+p},\ p > z$$

was chosen to add damping to the system and to increase the bandwidth. A MATLAB script was written to automate the search for a feasible zero-pole pair (z,p) located in the Appendix. The optimal values found are:

$$z = 9.479, \qquad p = 3690.526, \qquad K = 10^6$$

Therefore, the optimal lead compensator is:

$$G_c(s) = 10^6 \frac{s+9.479}{s+3690.526}$$

This satisfies our previous Root Locus Analysis constraints of the closed loop poles being to the left of $-8\ s^{-1}$. However, when analyzing the closed loop poles, one of these poles violates the angle constraint which is quite puzzling.

For this design, with the simulated 10° unit-step response, the overshoot is approximately 1.0%, and the settling time is ~0.0024s, with a tracking error of 1.66×10^{-4} rad which satisfies all requirements without needing a lag compensator.

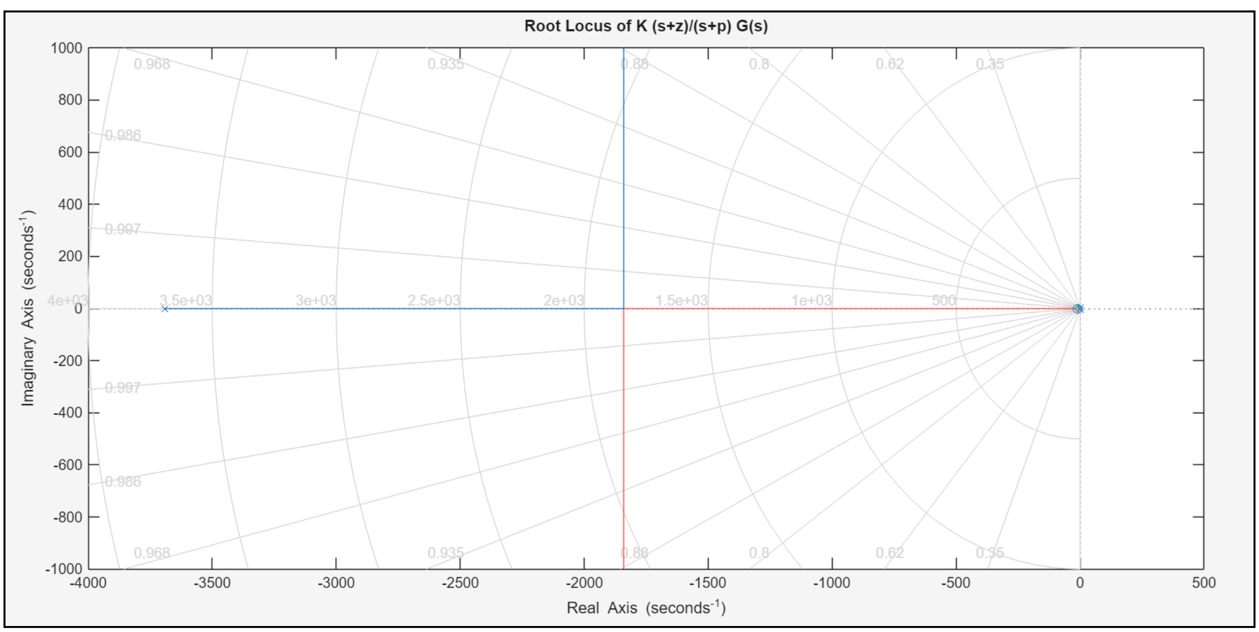

Figure 3: Root Locus

Figure 3 shows the root locus of the linearized pendulum system with the designed lead compensator applied. The closed-loop pole locations confirm that the compensator effectively increases the damping and shifts the poles to achieve convergence within the given constraints. Additionally, the branches run off of the screen due to scaling errors with the y-axis. For clarity, these branches stop at approximately ~$\pm$2.21e5 j s^{-1}.

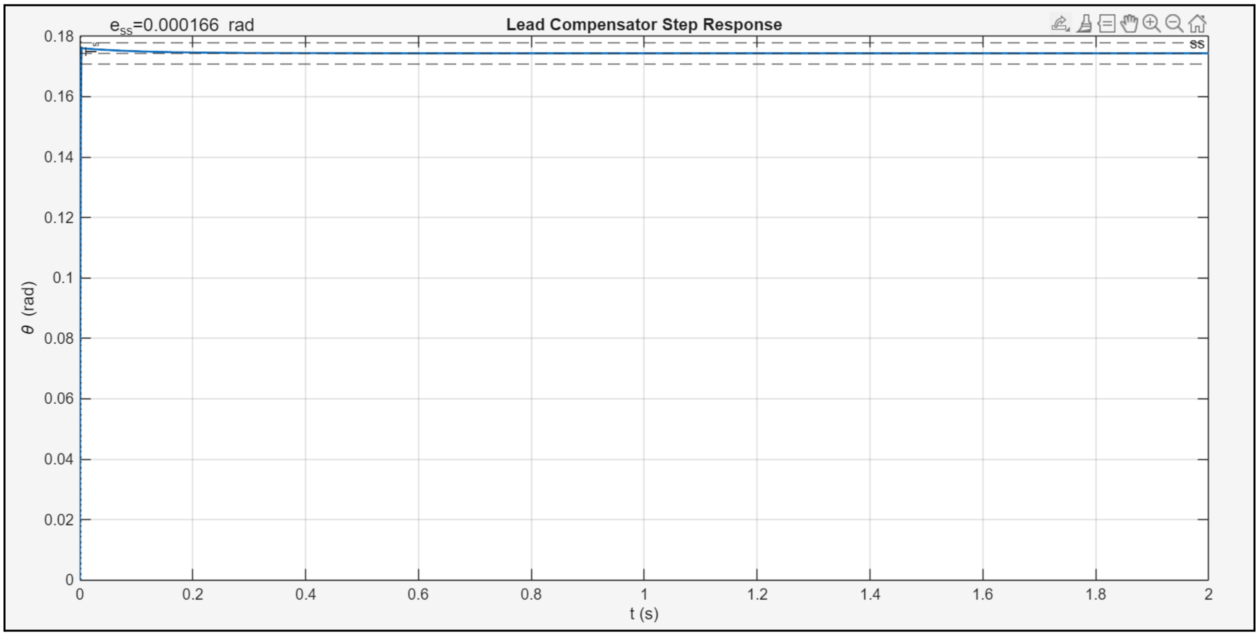

Figure 4: Lead Compensator Step Response

Figure 4 shows the corresponding unit-step response, which shows the quick settling time, small overshoot, and low steady-state response error from our designed lead compensator.

5. Linear Simulink Implementation

To validate the designed controller on the linearized pendulum model, the transfer function found in part 3 and the lead compensator designed in part 4 were implemented in Simulink using a standard unity-feedback configuration.

5.1 Linear Simulink Setup

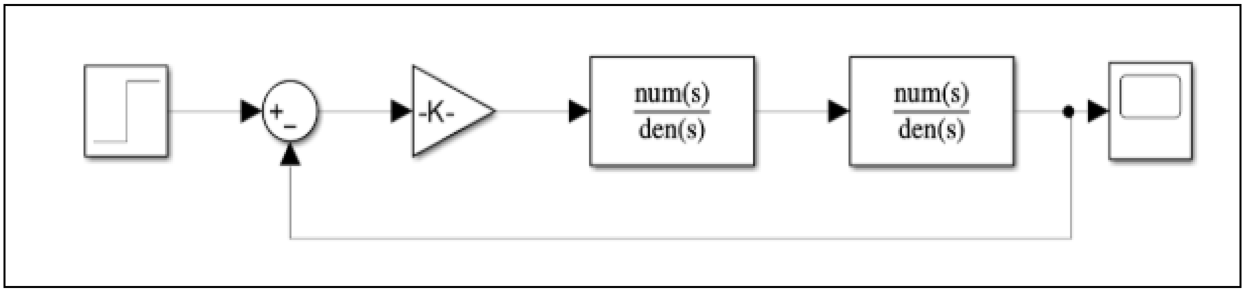

Figure 5: Linear Simulink Implementation

The block diagram shown in figure 5 consists of the step input block that generates a 10° command input, the sum block which computes the tracking error, the gain block which is part of the feedback loop to define unity negative feedback. The two transfer function blocks are the transfer function of the linearized pendulum system, and the designed lead compensator from parts 3 and 4 respectively. Finally, the scope block records the output $\theta(t)$.

5.2 Linear Response Results

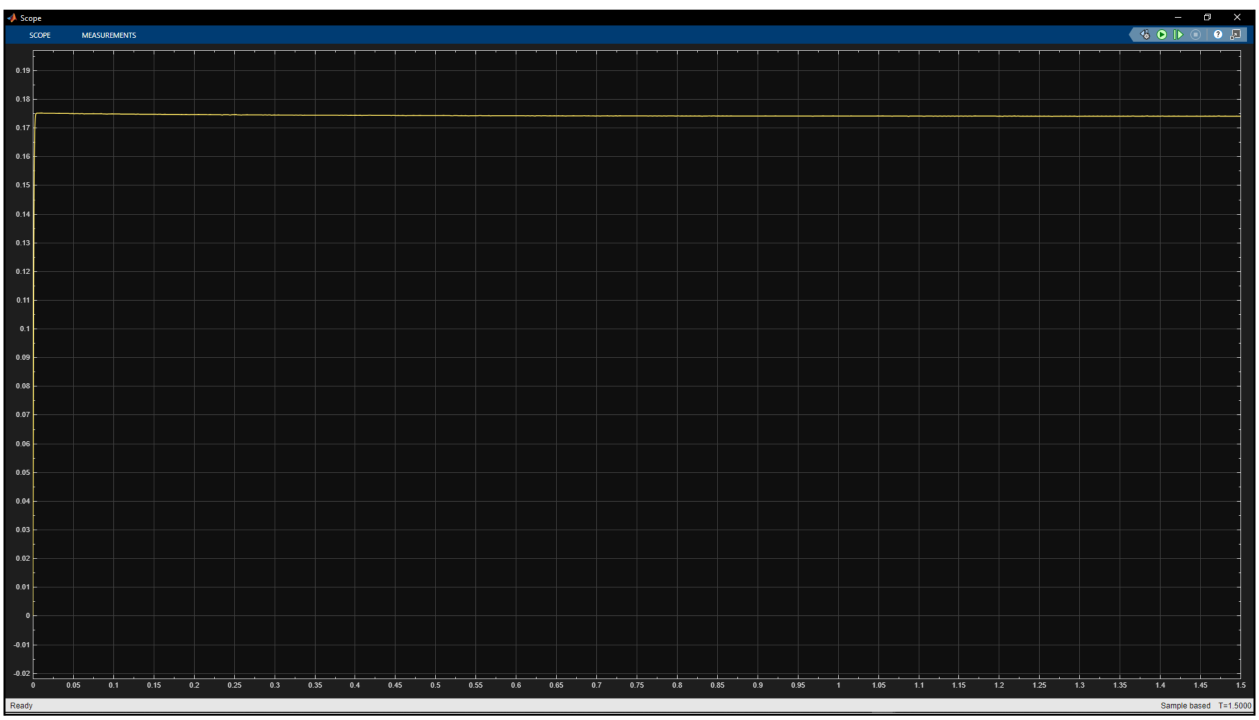

Figure 6: Linear Simulink Output

Figure 6 shows the output of the linear Simulink implementation. The output shows that the system quickly rises to the desired value and has negligible steady-state error. From the graph shown, the system reaches steady-state within 0.002 seconds which matches our predicted settling time found with MATLAB. The overshoot is less than 1%, much lower than the <5% requirement, and the steady-state error is essentially 0, meaning the high compensator gain and zero/pole placement provides sufficient damping. The curve itself appears flat after the initial rise, meaning the damping is very strong. These results show that the designed controller is effective before moving to the nonlinear Simulink diagram.

6. Nonlinear Simulink Implementation

The final stage of this project was to test the lead compensator on the full nonlinear pendulum dynamics. The linearized model is only valid for small deviations around the equilibrium point, the nonlinear model takes into account the gravitational torque, and nonlinear motion and verifies whether the designed controller remains effective under those conditions.

6.1 Nonlinear Simulink Setup

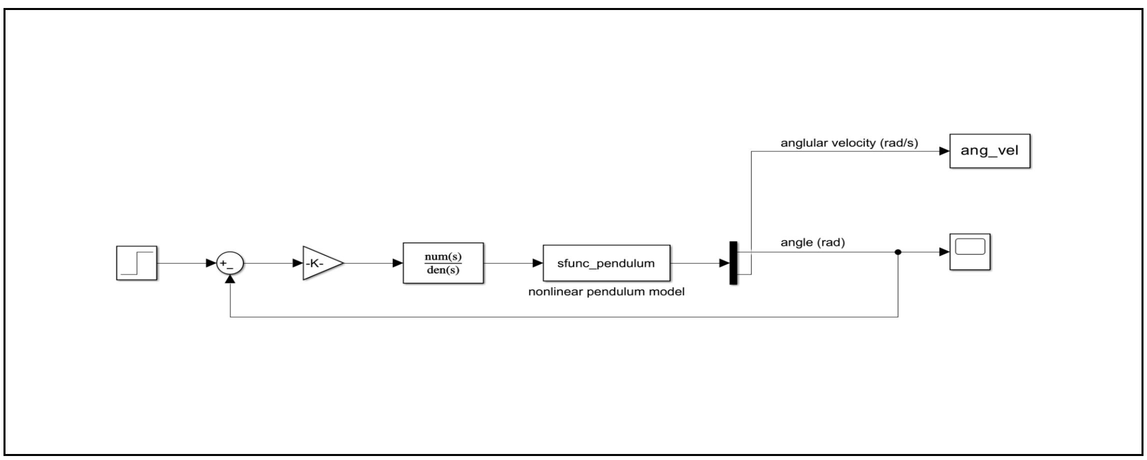

Figure 7: Nonlinear Simulink Implementation

The nonlinear pendulum system was set up using the provided S-function block which computes the full nonlinear equations of motion, and the compensator output is added to the required equilibrium torque. The Simulink model consists of the step input block for the 10° command, the sum block which computes the tracking error. The step block, and sum block are the same for both linear and nonlinear systems. The lead compensator transfer function block, the S-Function nonlinear plant block, which implements the true nonlinear pendulum characteristics, the Demux block, which separates the angle and angular velocity outputs, and two scope blocks. One for the angle output, and the other for the angular velocity output.

6.2 Nonlinear Response Results

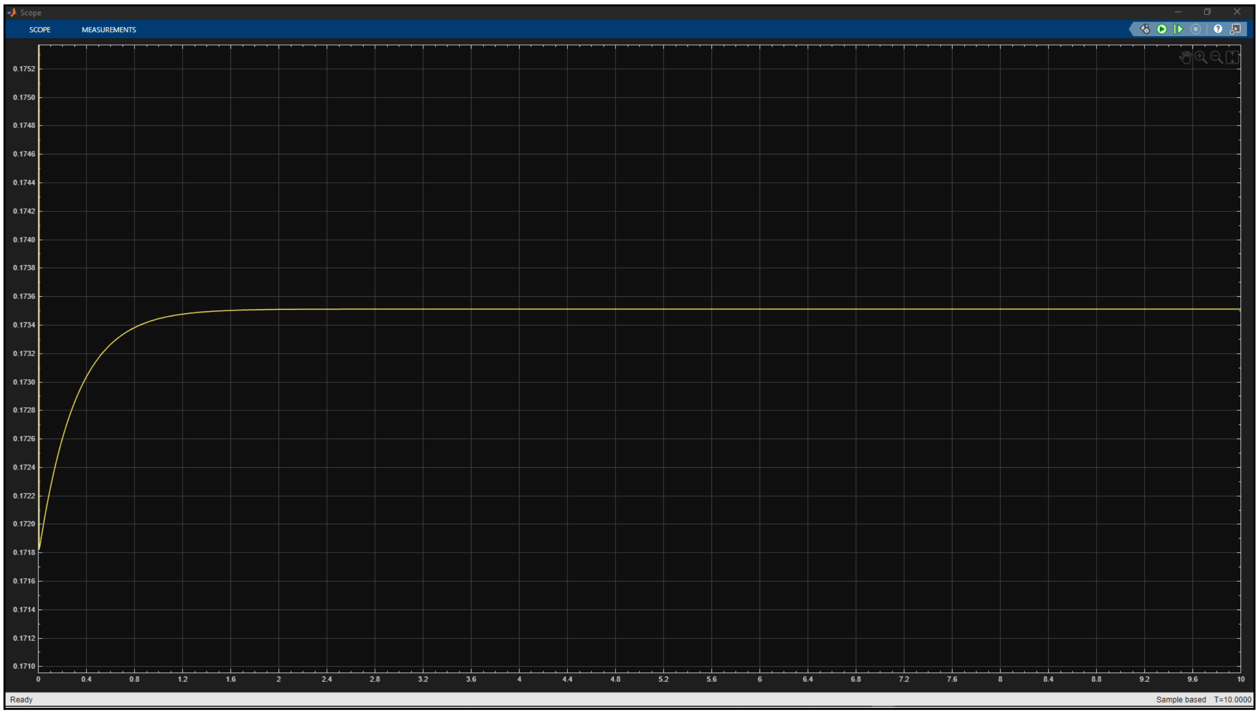

Figure 8: Nonlinear Simulink Output (Angle)

Figure 8 shows a smooth, well-damped rise to its final value. The initial value is located at $\theta(0) = \frac{\pi}{3} = 1.0472\ rad$ while the commanded value is 10° or 0.1745 rad which explain the heavy dip at the very beginning. This means that the controller is producing a large negative control torque at the beginning, to account for the large negative error, which forces the pendulum to rotate downward. Once the pendulum begins nearing the commanded angle, the error becomes smaller, and the controller output transitions to a much smoother, well-damped motion. Once the controller adjusts for the large error at the beginning, the output rises to the desired angle with almost 0 steady-state error, zero overshoot, 0.2 second settling time, and monotonic convergence, showing strong damping from the lead compensator. Although the nonlinear system includes a sin(θ) term, the output nearly matches our linear predictions, which shows the effective gain and phase compensation.

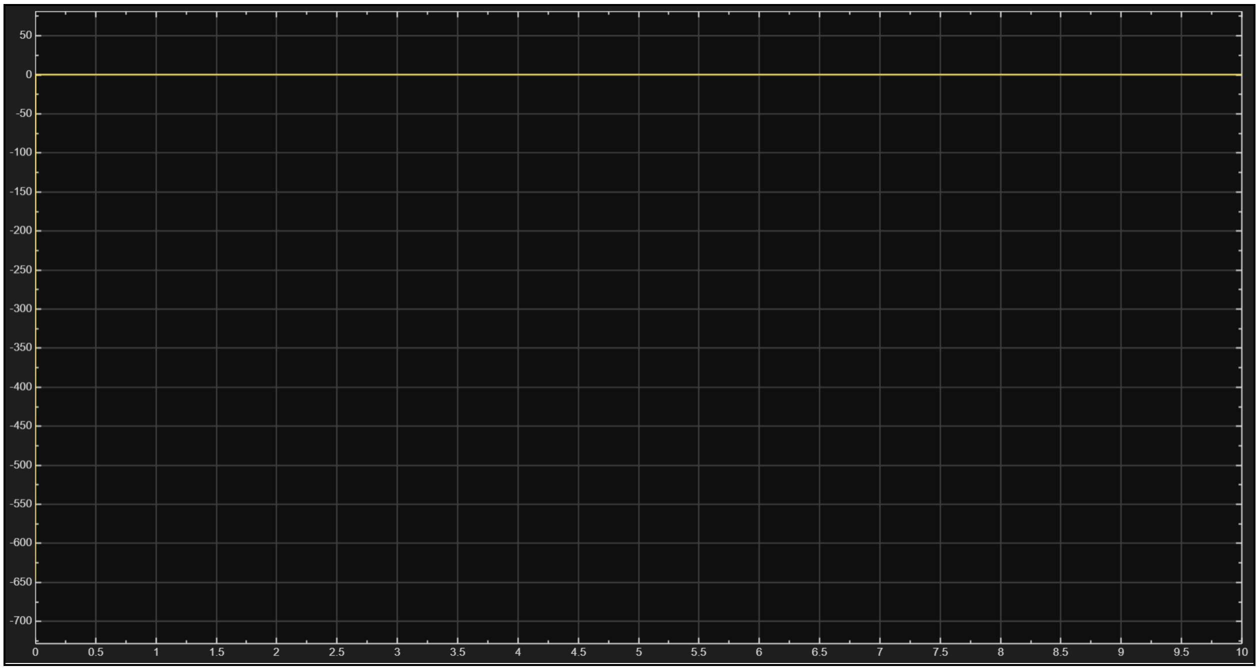

Figure 9: Nonlinear Simulink Output (Angular Velocity)

Figure 9 further reinforces the stability and damping of the nonlinear closed-loop system. After the initial transient, the angular velocity decays to zero very quickly, and remains almost constant at 0. The peak magnitude is small and short-lived, and reaches zero in about 0.1 seconds. There is no oscillation. This shows that the compensator successfully drives the pendulum to its commanded equilibrium angle while suppressing the nonlinear tendency to oscillate.

6.3 SIMULINK Notes

There were a few obstacles with using SIMULINK. Firstmost, the linear system was solved using ode45 solver (Model Settings) while the nonlinear system was solved using the ode15 solver, using alternative solvers could potentially cause chaotic, inaccurate responses or even no response at all (crash). Moreover, due to the extremely high value of K, if K were included in the numerator of the transfer function block, the SIMULINK would not execute. Therefore, a separate gain block was utilized. Lastly, when analyzing the response for the nonlinear system with a step input of 10°, the response starts abruptly at 60° and exactly at x = 1e-2, it transitions downwards into the curve as seen in

Figure 8. This is attributed to the initial position coded into the 'sfunc' block. Moreover, when checking Model Settings, there is a step size section which easily explains this issue.

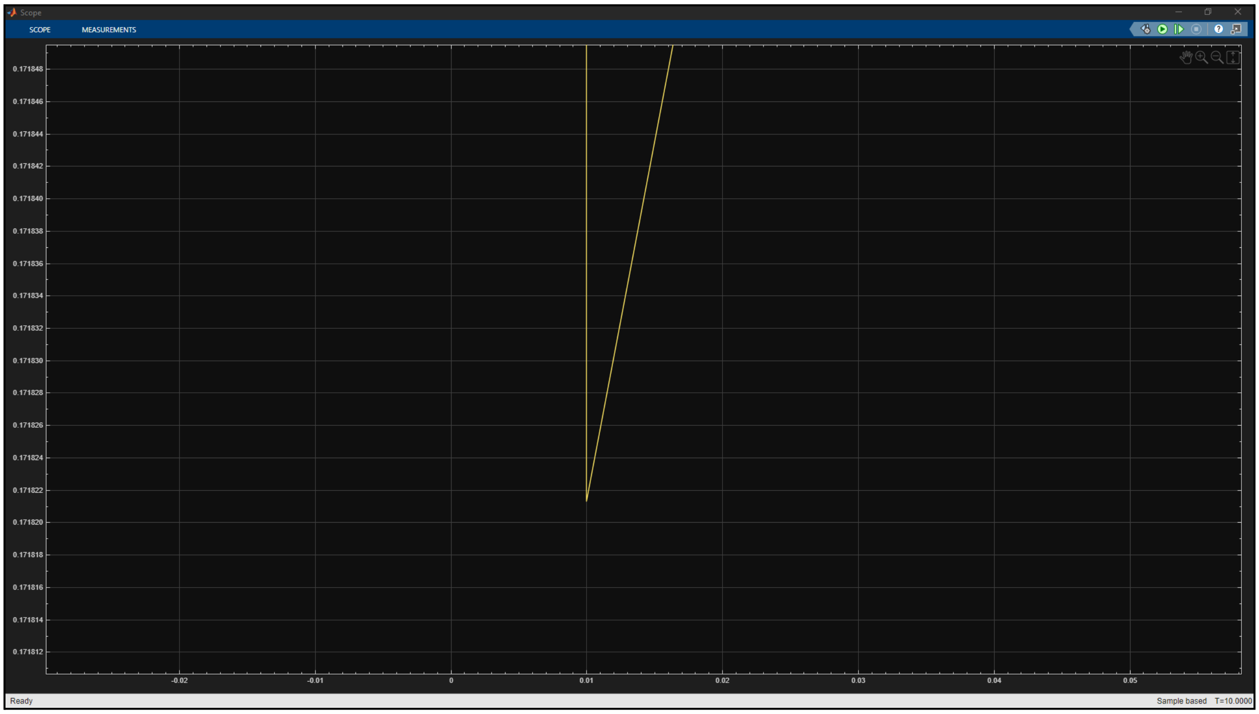

Figure 10: Discretization Error in the Scope Response

7. Discussion

The results of this project demonstrate that the linearization model accurately predicts the nonlinear behavior of the pendulum and provides a reliable foundation for controller design. As expected, the open-loop system exhibited undampened oscillations and could not meet any of the transient or steady state requirements outlined in the project, confirming the need for a compensator. The lead controller was designed using the root locus method and introduced significant damping while also increasing the accuracy of the system (negligible error). This caused the closed-loop poles to shift far left in the complex plane, and conceptually this makes sense: in class we are taught that the farther left a pole is, the more transient its response. Conversely, the closer to the imaginary axis a pole is, the more 'dominant' or conductive to steady state its response is. This resulted in a significantly short rise time, and extremely low overshoot, and a minute tracking error.

Both the linear and nonlinear simulations confirmed that the controller design remained effective even when applied to nonlinear systems. Although the nonlinear system includes gravity and is overall more complex, the compensator still managed to achieve negligible overshoot and almost zero steady state error for the step unit input. This performance remained consistent across both models, signaling that the linearization technique was appropriate for the chosen position. The only trade off is that the extremely large compensator gain may be more aggressive than needed in real world applications, but within theoretical simulation it provided high performance markers.

8. Conclusion

This project successfully designed and validated a lead compensator capable of regulating a rotary pendulum at the non-zero equilibrium angle of $\Theta = \frac{\pi}{3}$. The control design began by linearizing the nonlinear dynamics around this position, yielding a second order undampened model. Analyzing the open loop system confirmed inherent oscillation, warranting the necessity of compensation. Using the root locus method, the compensator was designed to meet strict performance markers, including less than 5% overshoot, a settling time of under 0.5 second, and a steady state error below 0.01°.

The final lead compensator substantially outperformed these requirements, achieving a very fast settling time and overshoot. Additionally, the overshoot was less than the stringent requirement of 0.01°, meaning no need for a lag compensator. Validation across both linear and nonlinear SIMULINK simulations confirms the high performance of our compensator. Critically, the compensator designed using the linear approximation proved highly effective under the nonlinear dynamics near the operating position. Overall, this work demonstrates the practical and effective control systems engineering workflow: deriving a simplified model, applying linear techniques, and validating performance against the nonlinear behavior to precisely regulate a complex, nonlinear system.

9. Appendix

```
%% MAE 435 Project
% Professor: Fen Wu
% Names: Austin Chung, Nawedullah Akbari, Nathaniel McIntire
% Date: December 2, 2025
clear; clc; close all;
%% Simulation of Unit-Step Response
clear; clc; close all;
s = tf('s');
% Open-loop transfer function
G = 4/(s^2 + 9.81);
% Time vector for simulation
t = linspace(0, 5, 2000);
% Unit step in torque
[y, t] = step(G, t);
% Steady-state value
theta_ss = mean(y(end-50:end));
fprintf('Steady-state angle ≈ %.6f rad (%.2f deg)\n', ...
       theta_ss, theta_ss*180/pi);
%% Plot
figure;
plot(t, y, 'LineWidth', 1.5);
grid on;
xlabel('Time (s)');
ylabel('\tilde\theta(t)  (rad)');
title('Unit-Step Response: \tilde T_c = 1 N·m');
yline(theta_ss,'--','\theta_{ss}','LabelHorizontalAlignment','left');
%% Lead Compensation via Root Locus Method
s = tf('s');
G = 4/(s^2 + 9.81);
u = pi/18;
% search ranges
Z    = linspace(0.1, 10, 20);
P    = linspace(10, 1e4, 20);
Kmax = 1e6;
t    = linspace(0,2,2500);
% performance limits
```

```
OS_max  = 0.05;
Ts_max  = 0.5;
ess_max = (pi/18)*1e-3;
bestCost  = Inf;
haveFound = false;
for z = Z
   for p = P
       if p <= z, continue; end
       L0 = (s+z)/(s+p)*G;
       for K = linspace(500, Kmax, 25)
           T = feedback(K*L0,1);
           poles = pole(T);
           if any(real(poles)>=0)
               continue
           end
           y = step(u*T, t);
           if any(~isfinite(y)), continue; end
           final = mean(y(end-30:end));
           if abs(final) < 1e-9
               continue;
           end
           peak = max(y);
           % Overshoot
           OS = max(0,(peak-final)/abs(final));
           % 2% settling time
           band = 0.02*abs(final);
           err  = abs(y-final);
           Ts   = t(end);
           for i = 1:length(err)
               if all(err(i:end) <= band)
                   Ts = t(i); break;
               end
           end
           % reject designs that don't meet specs
           if OS > OS_max || Ts > Ts_max
               continue;
           end
           % steady-state tracking error
```

```
            ess = abs(u - final);
            if ess > ess_max
                % fails steady-state requirement (this is what you'd
                % later fix with a lag compensator if needed)
                continue;
            end
            % cost among feasible designs
            cost = 40*OS + Ts;
            if cost < bestCost
                bestCost = cost;  haveFound = true;
                bz = z;   bp = p;   bK = K;
                bOS = OS; bTs = Ts; bfinal = final;
                by  = y;  bess = ess;
            end
        end
    end
end
if ~haveFound
    error('No design in the search grid meets all specs. Try
widening/adjusting the ranges.');
end
fprintf("\nBEST FEASIBLE DESIGN:\n");
fprintf("z = %.3f\np = %.3f\nK = %.0f\n", bz, bp, bK);
fprintf("OS = %.2f %%\n", bOS*100);
fprintf("Ts = %.4f s\n", bTs);
fprintf("steady-state value = %.6f rad\n", bfinal);
fprintf("tracking error e_ss = %.6e rad (limit = %.2e)\n", bess,
ess_max);
%% Step Response Plot
figure;
plot(t, by, 'LineWidth', 1.4); hold on;
yline(bfinal,'--','ss');
yline(bfinal*1.02,'--');
yline(bfinal*0.98,'--');
xline(bTs,':','T_s');
text(0.05, bfinal*1.05, sprintf('e_{ss}=%.3g rad', bess), 'FontSize',
12);
title('Lead Compensator Step Response');
```

```
xlabel('t (s)');
ylabel('\theta (rad)');
grid on;
%% Root Locus
figure;
rlocus((s+bz)/(s+bp)*G);
title('Root Locus of K (s+z)/(s+p) G(s)');
grid on;
```

Nathan McIntire | MAE 435 | Project

Part one

Derive the linearized state-space model with required torque

$$\begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \end{bmatrix} = \begin{bmatrix} x_2 \\ -\frac{g}{\ell}\sin(x_1) + \frac{T_c}{m\ell^2} \end{bmatrix}$$ ← Nonlinear state-space equation of the simple pendulum

$y = x_1$ ← output

$x_1 = \theta$ (pendulum angle, rad)

$x_2 = \dot{\theta}$ (angular velocity, rad/s)

input $u = T_c$ (control torque, N·m)

Output $y = \theta$

$$\dot{x} = f(x,u), \quad y = g(x,u)$$

$$f(x,u) = \begin{bmatrix} f_1(x,u) \\ f_2(x,u) \end{bmatrix} = \begin{bmatrix} x_2 \\ -\frac{g}{\ell}\sin(x_1) + \frac{u}{m\ell^2} \end{bmatrix}, \quad g(x,u) = x_1$$

$m = 1$ kg

$\ell = 0.5$ m

$g = 9.81$ m/s²

Find Equilibrium $(\bar{x}, \bar{u})$

- desired angle: $\bar{\theta} = \frac{\pi}{3}$
- desired angular velocity: $\bar{\dot{\theta}} = 0$

$$\bar{x} = \begin{bmatrix} \bar{x}_1 \\ \bar{x}_2 \end{bmatrix} = \begin{bmatrix} \bar{\theta} \\ 0 \end{bmatrix} = \begin{bmatrix} \pi/3 \\ 0 \end{bmatrix}$$

Set $\dot{x}_1 = 0$ and $\dot{x}_2 = 0$, solve for $\bar{u} = \bar{T}_c$

$$0 = \dot{x}_1 = \bar{x}_2 = 0$$

$$0 = \dot{x}_2 = -\frac{g}{\ell}\sin(\bar{x}_1) + \frac{\bar{u}}{m\ell^2}$$

$$\bar{x}_1 = \bar{\theta} = \frac{\pi}{3}$$

$$0 = -\frac{g}{\ell}\sin\left(\frac{\pi}{3}\right) + \frac{\bar{T}_c}{m\ell^2}$$

$$\frac{\bar{T}_c}{m\ell^2} = \frac{g}{\ell}\sin\left(\frac{\pi}{3}\right)$$

$(\bar{x}, \bar{u})$ is equilibrium if

$f(\bar{x}, \bar{u}) = 0 \Rightarrow \dot{x} = 0$

$$\bar{T}_c = \frac{g m \ell^2}{\cancel{\ell}}\sin\frac{\pi}{3}$$

$$\bar{T}_c = g m \ell \sin\left(\frac{\pi}{3}\right)$$

$$\bar{T}_c = 9.81(1)(0.5)\sin\frac{\pi}{3}$$

$$\bar{u} = \bar{T}_c = 4.248 \text{ N·m}$$

$$\bar{y} = g(\bar{x}, \bar{u}) = \bar{x}_1 = \bar{\theta} = \frac{\pi}{3}$$

Linearization continued.

$$x = \bar{x} + \tilde{x} \Rightarrow \tilde{x} = x - \bar{x}$$
$$\tilde{x}_1 = \theta - \bar{\theta}$$
$$\tilde{x}_2 = \dot{\theta} - 0 = \dot{\theta}$$
$$u = \bar{u} + \tilde{u} = \bar{T}_c + \tilde{T}_c$$
$$y = \bar{y} + \tilde{y} \Rightarrow \tilde{y} = y - \bar{y} = \theta - \bar{\theta}$$
$$\dot{\tilde{x}} = A\tilde{x} + B\tilde{u}, \quad \tilde{y} = C\tilde{x} + D\tilde{u} \leftarrow \text{goal}$$

$$f(x,u) \simeq f(\bar{x},\bar{u}) + \frac{\partial f}{\partial x}\Big|_{(\bar{x},\bar{u})}(x - \bar{x}) + \frac{\partial f}{\partial u}\Big|_{(\bar{x},\bar{u})}(u - \bar{u})$$

@ Equilibrium, $f(\bar{x},\bar{u}) = 0$

$$\dot{x} = f(x,u) \simeq A\tilde{x} + B\tilde{u}$$
$$A = \frac{\partial f}{\partial x}\Big|_{(\bar{x},\bar{u})}$$
$$B = \frac{\partial f}{\partial x}\Big|_{(\bar{x},\bar{u})}$$

$$g(x,u) \simeq g(\bar{x},\bar{u}) + \frac{\partial g}{\partial x}\Big|_{(\bar{x},\bar{u})}(x - \bar{x}) + \frac{\partial g}{\partial u}\Big|_{(\bar{x},\bar{u})}(u - \bar{u})$$

$$y = g(x,u) \simeq \bar{y} + C\tilde{x} + D\tilde{u}$$
$$\tilde{y} = y - \bar{y} \simeq C\tilde{x} + D\tilde{u}$$
$$C = \frac{\partial y}{\partial x}\Big|_{(\bar{x},\bar{u})}$$
$$D = \frac{\partial g}{\partial u}\Big|_{(\bar{x},\bar{u})}$$

$$f_1(x,u) = x_2$$
$$f_2(x,u) = -\frac{g}{\ell}\sin(x_1) + \frac{u}{m\ell^2}$$
$$A = \frac{\partial f}{\partial x} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} \\ \frac{\partial f_2}{\partial x_1} & \frac{\partial f_2}{\partial x_2} \end{bmatrix}$$

$\frac{\partial f_1}{\partial x_1}$ $\quad f_1 = x_2$, does not depend on x_1

$$\frac{\partial f_1}{\partial x_1} = 0$$
$$\frac{\partial f_1}{\partial x_2} = 1$$
$$\frac{\partial f_2}{\partial x_1} = \frac{\partial}{\partial x_1}\left[-\frac{g}{\ell}\sin(x_1)\right] = -\frac{g}{\ell}\cos(x_1)$$
$$\frac{\partial f_2}{\partial x_2} = 0$$

$$A = \begin{bmatrix} 0 & 1 \\ -\frac{g}{\ell}\cos(x_1) & 0 \end{bmatrix}$$

evaluate @ $x_1 = \bar{\theta}$, $x_2 = 0$, $u = \bar{T}_c$

$$A = A(\bar{x},\bar{u}) = \begin{bmatrix} 0 & 1 \\ -\frac{g}{\ell}\cos(\bar{\theta}) & 0 \end{bmatrix}$$

Nathan McIntosh | MAE 435 | Project

$$B = \frac{\partial f}{\partial u} : \begin{bmatrix} \frac{\partial f_1}{\partial u} \\ \frac{\partial f_2}{\partial u} \end{bmatrix}$$

$f_1 : x_2 \rightarrow$ independent of u

$$\frac{\partial f_1}{\partial u} : 0$$

$$f_2 : -\frac{g}{\ell}\sin(x_1) + \frac{u}{m\ell^2}$$

$$\frac{\partial f_2}{\partial u} : \frac{1}{m\ell^2}$$

$$B = \begin{bmatrix} 0 \\ 1/m\ell^2 \end{bmatrix}$$

$$g(x,u) = x_1 = g(x_1, x_2, u)$$

$$C : \frac{\partial g}{\partial x} = \begin{bmatrix} \frac{\partial g}{\partial x_1} & \frac{\partial g}{\partial x_2} \end{bmatrix}$$

$$\frac{\partial g}{\partial x_1} = 1 \qquad g = x_1$$

$$\frac{\partial g}{\partial x_2} = 0$$

$$C = \begin{bmatrix} 1 & 0 \end{bmatrix}$$

$$D = \frac{\partial g}{\partial u} = 0$$

$g = 9.81$

$\ell = 0.5$

$M = 1$

$\bar{\theta} = \pi/3$

$$A = \begin{bmatrix} 0 & 1 \\ -g/\ell\cos\bar{\theta} & 0 \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ -\frac{9.81}{0.5}\cos\frac{\pi}{3} & 0 \end{bmatrix}$$

$$A = \begin{bmatrix} 0 & 1 \\ -9.81 & 0 \end{bmatrix}$$

$$B = \begin{bmatrix} 0 \\ \frac{1}{M\ell^2} \end{bmatrix} = \begin{bmatrix} 0 \\ \frac{1}{1(0.5)^2} \end{bmatrix} = \begin{bmatrix} 0 \\ 4 \end{bmatrix}$$

$$C = \begin{bmatrix} 1 & 0 \end{bmatrix}$$

$$D = 0$$

$$\dot{\hat{x}} = \begin{bmatrix} 0 & 1 \\ -9.81 & 0 \end{bmatrix}\tilde{x} + \begin{bmatrix} 0 \\ 4 \end{bmatrix}\tilde{u}$$

$$\hat{x}_1 = \theta - \frac{\pi}{3} \qquad \hat{y} = \theta - \frac{\pi}{3}$$

$$\tilde{x}_2 = \dot{\theta} - 0 = \dot{\theta}$$

$$\hat{y} = \begin{bmatrix} 1 & 0 \end{bmatrix}\tilde{x} + 0 \cdot \tilde{u}$$

$$\tilde{u} = T_c - \bar{T}_c \Rightarrow T_c - 4.246$$

TOPS

Nathan McIntire | MAE 4..

Part 2

TRANSFER FUNCTION

$$G(s) = \frac{\tilde{\Theta}(s)}{\tilde{T}_c(s)}$$

$$\dot{\tilde{x}} = \underbrace{\begin{bmatrix} 0 & 1 \\ -9.81 & 0 \end{bmatrix}}_{A} \tilde{x} + \underbrace{\begin{bmatrix} 0 \\ 4 \end{bmatrix}}_{B} \tilde{u}$$

$$\tilde{y} = \underbrace{\begin{bmatrix} 1 & 0 \end{bmatrix}}_{C} \tilde{x} \qquad D = 0$$

$$s\tilde{X}(s) = A\tilde{X}(s) + B\tilde{U}(s)$$

$$\Rightarrow (sI - A)\tilde{X}(s) = B\tilde{U}(s)$$

$$\Rightarrow \tilde{X}(s) = (sI - A)^{-1} B\tilde{U}(s)$$

← zero initial conditions Laplace Transform

Output

$$\tilde{Y}(s) = C\tilde{X}(s) + D\tilde{U}(s) = C(sI-A)^{-1}B\tilde{U}(s) + \xrightarrow{0}\cancel{D\tilde{U}(s)}$$

$$\tilde{Y}(s) = C(sI - A)^{-1} B\tilde{U}(s)$$

$$G(s) = \frac{\tilde{Y}(s)}{\tilde{U}(s)} = C(sI-A)^{-1}B$$

$sI - A$

$$sI = \begin{bmatrix} s & 0 \\ 0 & s \end{bmatrix} - \begin{bmatrix} 0 & 1 \\ -9.81 & 0 \end{bmatrix} = \begin{bmatrix} s & -1 \\ 9.81 & s \end{bmatrix}$$

$$(sI - A)^{-1} \rightarrow \frac{1}{ad - bc} = \frac{1}{s^2 + 9.81} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix}$$

$$= \frac{1}{s^2 + 9.81} \begin{bmatrix} s & 1 \\ -9.81 & s \end{bmatrix}$$

Multiply by B

$$\frac{1}{s^2+9.81} \begin{bmatrix} s & 1 \\ -9.81 & s \end{bmatrix} \begin{bmatrix} 0 \\ 4 \end{bmatrix} = \frac{1}{s^2+9.81} \begin{bmatrix} 4 \\ 4s \end{bmatrix}$$

Nathan McEntire | MAE 435

Multiply by C

$$\left(\frac{1}{s^2+9.81}\begin{bmatrix}4\\4s\end{bmatrix}\right)\begin{bmatrix}1 & 0\end{bmatrix} = \frac{1}{s^2+9.81}(1\cdot 4 + 0\cdot 4s)$$

$$= \boxed{\frac{4}{s^2+9.81}}$$